

PyTorch RNN from Scratch

11 minute read

In this post, we'll take a look at RNNs, or recurrent neural networks, and attempt to implement parts of it in scratch through PyTorch. Yes, it's not entirely from scratch in the sense that we're still relying on PyTorch autograd to compute gradients and implement backprop, but I still think there are valuable insights we can glean from this implementation as well.

For a brief introductory overview of RNNs, I recommend that you check out [this previous post \(https://jaketae.github.io/study/rnn/\)](https://jaketae.github.io/study/rnn/), where we explored not only what RNNs are and how they work, but also how one can go about implementing an RNN model using Keras. This time, we will be using PyTorch, but take a more hands-on approach to build a simple RNN from scratch.

Full disclaimer that this post was largely adapted from [this PyTorch tutorial \(https://pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html\)](https://pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html) this PyTorch tutorial. I modified and changed some of the steps involved in preprocessing and training. I still recommend that you check it out as a supplementary material. With that in mind, let's get started.

Data Preparation

The task is to build a simple classification model that can correctly determine the nationality of a person given their name. Put more simply, we want to be able to tell where a particular name is from.

Download

We will be using some labeled data from the PyTorch tutorial. We can download it simply by typing

```
curl -O https://download.pytorch.org/tutorial/data.zip; unzip data.zip
```

This command will download and unzip the files into the current directory, under the folder name of `data`.

Now that we have downloaded the data we need, let's take a look at the data in more detail. First, here are the dependencies we will need.

```
import os
import random
from string import ascii_letters

import torch
from torch import nn
import torch.nn.functional as F
from unicode import unicode

_ = torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

We first specify a directory, then try to print out all the labels there are. We can then construct a dictionary that maps a language to a numerical label.

```
data_dir = "./data/names"

lang2label = {
    file_name.split(".")[0]: torch.tensor([i], dtype=torch.long)
    for i, file_name in enumerate(os.listdir(data_dir))
}
```

We see that there are a total of 18 languages. I wrapped each label as a tensor so that we can use them directly during training.

```
lang2label

{'Czech': tensor([0]),
 'German': tensor([1]),
 'Arabic': tensor([2]),
 'Japanese': tensor([3]),
 'Chinese': tensor([4]),
 'Vietnamese': tensor([5]),
 'Russian': tensor([6]),
 'French': tensor([7]),
 'Irish': tensor([8]),
 'English': tensor([9]),
 'Spanish': tensor([10]),
 'Greek': tensor([11]),
 'Italian': tensor([12]),
 'Portuguese': tensor([13]),
 'Scottish': tensor([14]),
 'Dutch': tensor([15]),
 'Korean': tensor([16]),
 'Polish': tensor([17])}
```

Let's store the number of languages in some variable so that we can use it later in our model declaration, specifically when we specify the size of the final output layer.

```
num_langs = len(lang2label)
```

Preprocessing

Now, let's preprocess the names. We first want to use `unicode` to standardize all names and remove any acute symbols or the likes. For example,

```
unicode("Ślusarski")

'Slusarski'
```

Once we have a decoded string, we then need to convert it to a tensor so that the model can process it. This can first be done by constructing a `char2idx` mapping, as shown below.

```
char2idx = {letter: i for i, letter in enumerate(ascii_letters + " .,:;-'")}
num_letters = len(char2idx); num_letters

59
```

We see that there are a total of 59 tokens in our character vocabulary. This includes spaces and punctuations, such as ``,:;-'`. This also means that each name will now be expressed as a tensor of size `(num_char, 59)`; in other words, each character will be a tensor of size `(59)`. We can now build a function that accomplishes this task, as shown below:

```
def name2tensor(name):
    tensor = torch.zeros(len(name), 1, num_letters)
    for i, char in enumerate(name):
        tensor[i][0][char2idx[char]] = 1
    return tensor
```

If you read the code carefully, you'll realize that the output tensor is of size `(num_char, 1, 59)`, which is different from the explanation above. Well, the reason for that extra dimension is that we are using a batch size of 1 in this case. In PyTorch, RNN layers expect the input tensor to be of size `(seq_len, batch_size, input_size)`. Since every name is going to have a different length, we don't batch the inputs for simplicity purposes and simply use each input as a single batch. For a more detailed discussion, check out this [forum discussion \(https://discuss.pytorch.org/t/batch-size-position-and-rnn-tutorial/41269/3\)](https://discuss.pytorch.org/t/batch-size-position-and-rnn-tutorial/41269/3).

Let's quickly verify the output of the `name2tensor()` function with a dummy input.

```
name2tensor("abc")
```

[illegible]

Dataset Creation

Now we need to build a our dataset with all the preprocessing steps. Let's collect all the decoded and converted tensors in a list, with accompanying labels. The labels can be obtained easily from the file name, for example `german.txt`

```

tensor_names = []
target_langs = []

for file in os.listdir(data_dir):
    with open(os.path.join(data_dir, file)) as f:
        lang = file.split('.')[0]
        names = [unicode(line.rstrip()) for line in f]
        for name in names:
            try:
                tensor_names.append(name2tensor(name))
                target_langs.append(target2label[lang])
            except KeyError:
                pass

```

We could wrap this in a PyTorch `dataset` class, but for simplicity sake let's just use a good old `for` loop to feed this data into our model. Since we are dealing with normal lists, we can easily use `sklearn`'s `train_test_split()` to separate the training data from the testing data.

```
from sklearn.model_selection import train_test_split

train_idx, test_idx = train_test_split(
    range(len(target_langs)),
    test_size=0.1,
    shuffle=True,
    stratify=target_langs
)

train_dataset = [
    (tensor_names[i], target_langs[i])
    for i in train_idx
]

test_dataset = [
    (tensor_names[i], target_langs[i])
    for i in test_idx
]
```

Let's see how many training and testing data we have. Note that we used a `test_size` of 0.1.

```
print(f"Train: {len(train_dataset)}")
print(f"Test: {len(test_dataset)}")
```

```
Train: 18063
Test: 2007
```

Model

We will be building two models: a simple RNN, which is going to be built from scratch, and a GRU-based model using PyTorch's layers.

Simple RNN

Now we can build our model. This is a very simple RNN that takes a single character tensor representation as input and produces some prediction and a hidden state, which can be used in the next iteration. Notice that it is just some fully connected layers with a sigmoid non-linearity applied during the hidden state computation.

```
class MyRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(MyRNN, self).__init__()
        self.hidden_size = hidden_size
        self.in2hidden = nn.Linear(input_size + hidden_size, hidden_size)
        self.in2output = nn.Linear(input_size + hidden_size, output_size)

    def forward(self, x, hidden_state):
        combined = torch.cat((x, hidden_state), 1)
        hidden = torch.sigmoid(self.in2hidden(combined))
        output = self.in2output(combined)
        return output, hidden

    def init_hidden(self):
        return nn.init.kaiming_uniform_(torch.empty(1, self.hidden_size))
```

We call `init_hidden()` at the start of every new batch. For easier training and learning, I decided to use `kaiming_uniform()` to initialize these hidden states.

We can now build our model and start training it.

```
hidden_size = 256
learning_rate = 0.001

model = MyRNN(num_letters, hidden_size, num_langs)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

I realized that training this model is very unstable, and as you can see the loss jumps up and down quite a bit. Nonetheless, I didn't want to cook my 13-inch MacBook Pro so I decided to stop at two epochs.

```

num_epochs = 2
print_interval = 3000

for epoch in range(num_epochs):
    random.shuffle(train_dataset)
    for i, (name, label) in enumerate(train_dataset):
        hidden_state = model.init_hidden()
        for char in name:
            output, hidden_state = model(char, hidden_state)
            loss = criterion(output, label)

        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1)
        optimizer.step()

    if (i + 1) % print_interval == 0:
        print(
            f"Epoch [{epoch + 1}/{num_epochs}], ",
            f"Step [{i + 1}/{len(train_dataset)}], ",
            f"Loss: {loss.item():.4f}"
        )

```

```

Epoch [1/2], Step [3000/18063], Loss: 0.0390
Epoch [1/2], Step [6000/18063], Loss: 1.0368
Epoch [1/2], Step [9000/18063], Loss: 0.6718
Epoch [1/2], Step [12000/18063], Loss: 0.0003
Epoch [1/2], Step [15000/18063], Loss: 1.0658
Epoch [1/2], Step [18000/18063], Loss: 1.0021
Epoch [2/2], Step [3000/18063], Loss: 0.0021
Epoch [2/2], Step [6000/18063], Loss: 0.0131
Epoch [2/2], Step [9000/18063], Loss: 0.3842
Epoch [2/2], Step [12000/18063], Loss: 0.0002
Epoch [2/2], Step [15000/18063], Loss: 2.5420
Epoch [2/2], Step [18000/18063], Loss: 0.0172

```

Now we can test our model. We could look at other metrics, but accuracy is by far the simplest, so let's go with that.

```

num_correct = 0
num_samples = len(test_dataset)

model.eval()

with torch.no_grad():
    for name, label in test_dataset:
        hidden_state = model.init_hidden()
        for char in name:
            output, hidden_state = model(char, hidden_state)
            _, pred = torch.max(output, dim=1)
            num_correct += bool(pred == label)

print(f"Accuracy: {num_correct / num_samples * 100:.4f}%")

```

```
Accuracy: 72.2471%
```

The model records a 72 percent accuracy rate. This is very bad, but given how simple the models is and the fact that we only trained the model for two epochs, we can lay back and indulge in momentary happiness knowing that the simple RNN model was at least able to learn something.

Let's see how well our model does with some concrete examples. Below is a function that accepts a string as input and outputs a decoded prediction.

```

label2lang = {label.item(): lang for lang, label in lang2label.items()}

def myrnn_predict(name):
    model.eval()
    tensor_name = name2tensor(name)
    with torch.no_grad():
        hidden_state = model.init_hidden()
        for char in tensor_name:
            output, hidden_state = model(char, hidden_state)
            _, pred = torch.max(output, dim=1)
    model.train()
    return label2lang[pred.item()]

```

I don't know if any of these names were actually in the training or testing set; these are just some random names I came up with that I thought would be pretty reasonable. And voila, the results are promising.

```
myrnn_predict("Mike")
```

```
'English'
```

```
myrnn_predict("Qin")
```

```
'Chinese'
```

```
myrnn_predict("Slaveya")
```

```
'Russian'
```

The model seems to have classified all the names into correct categories!

PyTorch GRU

This is cool and all, and I could probably stop here, but I wanted to see how this custom model fares in comparison to, say, a model using PyTorch layers. GRU is probably not fair game for our simple RNN, but let's see how well it does.

```

class GRUModel(nn.Module):
    def __init__(self, num_layers, hidden_size):
        super(GRUModel, self).__init__()
        self.num_layers = num_layers
        self.hidden_size = hidden_size
        self.gru = nn.GRU(
            input_size=num_letters,
            hidden_size=hidden_size,
            num_layers=num_layers,
        )
        self.fc = nn.Linear(hidden_size, num_langs)

    def forward(self, x):
        hidden_state = self.init_hidden()
        output, hidden_state = self.gru(x, hidden_state)
        output = self.fc(output[-1])
        return output

    def init_hidden(self):
        return torch.zeros(self.num_layers, 1, self.hidden_size).to(device)

```

Let's declare the model and an optimizer to go with it. Notice that we are using a two-layer GRU, which is already one more than our current RNN implementation.

```

model = GRUModel(num_layers=2, hidden_size=hidden_size)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

```

```

for epoch in range(num_epochs):
    random.shuffle(train_dataset)
    for i, (name, label) in enumerate(train_dataset):
        output = model(name)
        loss = criterion(output, label)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if (i + 1) % print_interval == 0:
        print(
            f"Epoch [{epoch + 1}/{num_epochs}], "
            f"Step [{i + 1}/{len(train_dataset)}], "
            f"Loss: {loss.item():.4f}"
        )

```

```

Epoch [1/2], Step [3000/18063], Loss: 1.8497
Epoch [1/2], Step [6000/18063], Loss: 0.4908
Epoch [1/2], Step [9000/18063], Loss: 1.0299
Epoch [1/2], Step [12000/18063], Loss: 0.0855
Epoch [1/2], Step [15000/18063], Loss: 0.0053
Epoch [1/2], Step [18000/18063], Loss: 2.6417
Epoch [2/2], Step [3000/18063], Loss: 0.0004
Epoch [2/2], Step [6000/18063], Loss: 0.0008
Epoch [2/2], Step [9000/18063], Loss: 0.1446
Epoch [2/2], Step [12000/18063], Loss: 0.2125
Epoch [2/2], Step [15000/18063], Loss: 3.7883
Epoch [2/2], Step [18000/18063], Loss: 0.4862

```

The training appeared somewhat more stable at first, but we do see a weird jump near the end of the second epoch. This is partially because I didn't use gradient clipping for this GRU model, and we might see better results with clipping applied.

Let's see the accuracy of this model.

```

num_correct = 0

model.eval()

with torch.no_grad():
    for name, label in test_dataset:
        output = model(name)
        _, pred = torch.max(output, dim=1)
        num_correct += bool(pred == label)

print(f"Accuracy: {num_correct / num_samples * 100:.4f}%")

```

```
Accuracy: 81.4150%
```

And we get an accuracy of around 80 percent for this model. This is better than our simple RNN model, which is somewhat expected given that it had one additional layer and was using a more complicated RNN cell model.

Let's see how this model predicts given some raw name string.

```

def pytorch_predict(name):
    model.eval()
    tensor_name = name2tensor(name)
    with torch.no_grad():
        output = model(tensor_name)
        _, pred = torch.max(output, dim=1)
    model.train()
    return label2lang[pred.item()]

```

```
pytorch_predict("Jake")
```

```
'English'
```

```
pytorch_predict("Qin")
```

```
'Chinese'
```

```
pytorch_predict("Fernando")
```

```
'Spanish'
```

```
pytorch_predict("Demirkan")
```

```
'Russian'
```

The last one is interesting, because it is the name of a close Turkish friend of mine. The model obviously isn't able to tell us that the name is Turkish since it didn't see any data points that were labeled as Turkish, but it tells us what nationality the name might fall under among the 18 labels it has been trained on. It's obviously wrong, but perhaps not too far off in some regards; at least it didn't say Japanese, for instance. It's also not entirely fair game for the model since there are many names that might be described as multi-national; perhaps there is a Russian person with the name of Demirkan.

Conclusion

I learned quite a bit about RNNs by implementing this RNN. It is admittedly simple, and it is somewhat different from the PyTorch layer-based approach in that it requires us to loop through each character manually, but the low-level nature of it forced me to think more about tensor dimensions and the purpose of having a division between the hidden state and output. It was also a healthy reminder of how RNNs can be difficult to train.

In the coming posts, we will be looking at sequence-to-sequence models, or seq2seq for short. Ever since I heard about seq2seq, I was fascinated by tthe power of transforming one form of data to another. Although these models cannot be realistically trained on a CPU given the constraints of my local machine, I think implementing them themselves will be an exciting challenge.

Catch you up in the next one!

Tags:

deep_learning

from_scratch

pytorch

Categories:

study

Updated: October 25, 2020